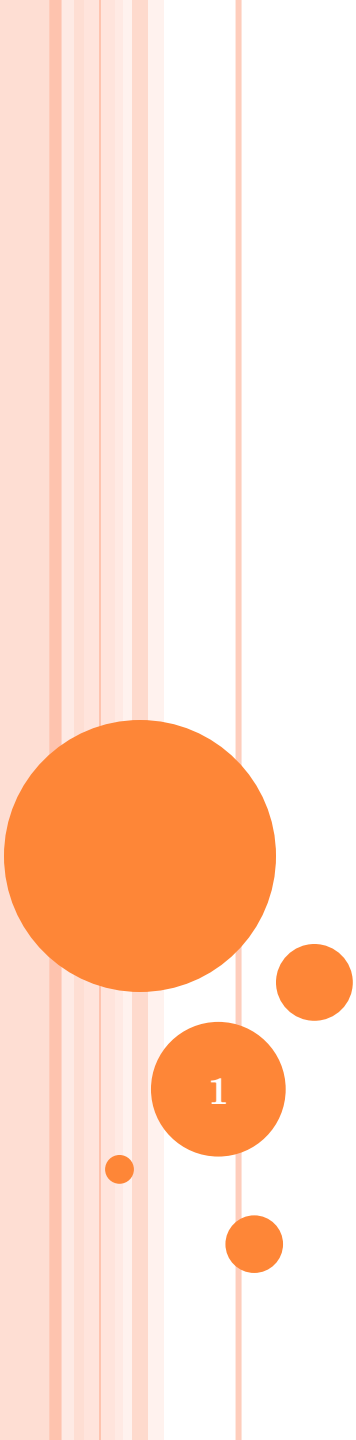




SISTEME DE OPERARE

Curs 3: Gestiunea dispozitivelor de stocare (partea 1)

Sistemul de fișiere

Kinga Márton

Universitatea Tehnică din Cluj-Napoca
Departamentul de Calculatoare

GESTIUNEA DISPOZITIVELOR DE STOCARE

○ În cuprins

- Sistem de fișiere (SF)
 - Definiții și concepte
 - Funcțiile SF
 - Interfața cu SF
 - Protecție
 - Niveluri
 - Implementare
- Concepte fundamentale
 - Fișiere – tipuri și attribute
 - Directoare - organizare
- Metode de acces
- Metode de alocare

CONTEXT - MEMORIA EXTERNĂ

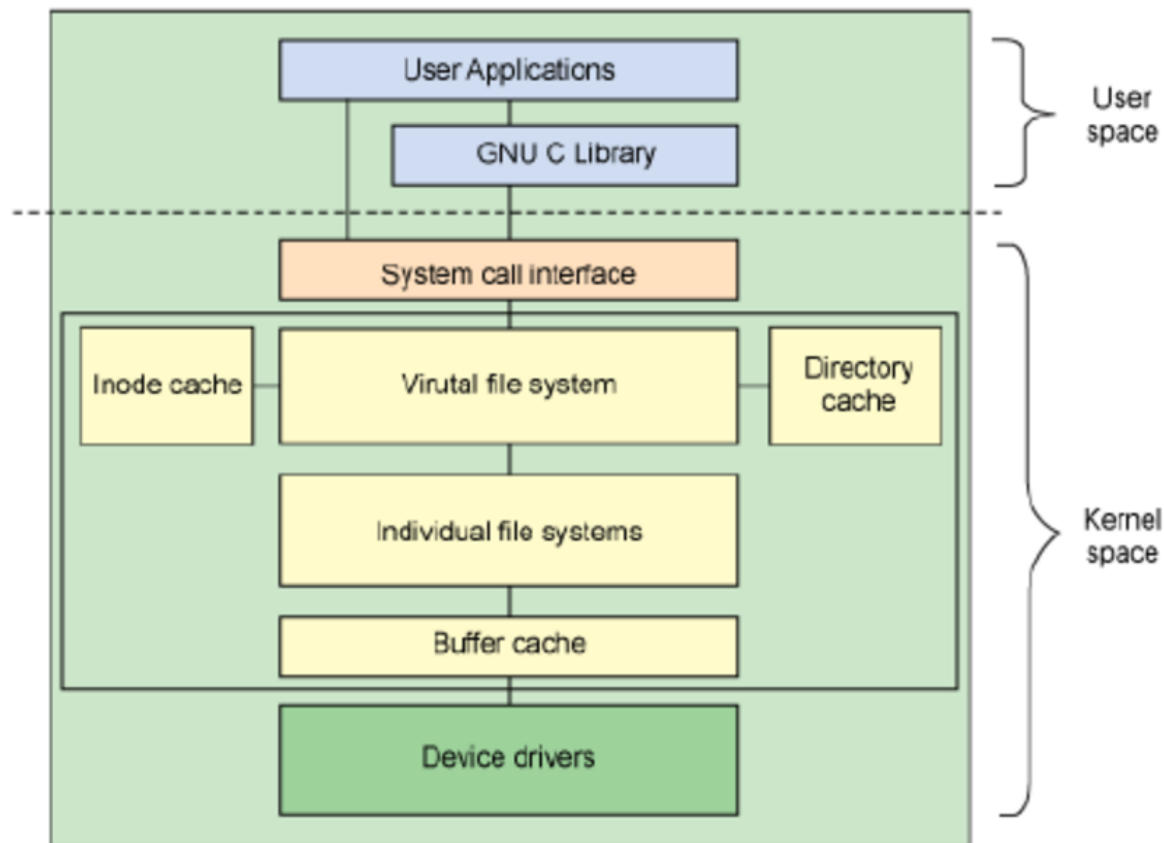
- Limitările memoriei interne
 - volatilă
 - dimensiuni relativ reduse
- Utilizatorii au nevoie să **stocheze** și să **partajeze** date și aplicații
 - De **dimensiuni mari**
 - În mod **persistent** (durabil)
- Soluție
 - **Memoria externă** – sistemul de Intrare/Ieșire (I/E)
 - Discuri magnetice, SSD, discuri optice, benzi magnetice, etc.
- Problemă
 - **Timp de acces** mare
 - Componenta cea mai lentă
 - **Diferite tipuri** de dispozitive de stocare – diferite caracteristici
- Provocare
 - **Organizarea eficientă** a informației stocate persistent în memoria externă
 - Furnizarea unei **interfețe simple** și eficiente
 - **Optimizarea operațiilor** de I/E pentru a permite concurență maximă

SISTEMUL DE FIȘIERE

- **Componenta SO** – cea mai vizibilă
- **Interfață** între utilizator și dispozitivele fizice de stocare
 - **Mapează** datele pe suport fizic
 - Furnizează **acces la informațiile** din spațiul de stocare
 - **Gestionează datele** stocate pe suportul extern
 - Prin structuri de date și algoritmi corespunzători
 - Interacționează direct cu dispozitivele de stocare
- Două părți distincte
 - **Colecție de fișiere**
 - Fiecare stochează date înrudite
 - **Structură de directoare**
 - Organizează și oferă informații despre fișierele din sistem
- **Rol**
 - Furnizarea unei viziuni logice uniforme asupra mediilor de stocare
 - Ascunde complexitatea și diversitatea dispozitivelor de stocare
 - SO introduce o **abstracțiune**
 - **Fișierul** – unitatea logică de stocare

STRUCTURA SISTEMULUI DE FIȘIERE

- Ascunde complexitatea dispozitivelor fizice de stocare
- Gestionează data pe suportul de stocare
 - Interacțiune directă cu hardware-ul
 - Are un nivel specific pt hardware-ul pe care rulează



FIȘIERUL

- Colecție de informații înrudite salvate pe memoria externă
- Unitatea logică de stocare
 - Dpdv. al utilizatorului fișierul este cea mai mică unitate logică de alocare a spațiului de stocare secundar
 - Datele pot fi scrise pe spațiul de stocare secundar doar în cadrul unor fișiere
- Tipuri de informații stocate în fișiere
 - Cod sursă, cod obiect, aplicații (executabile)
 - Date (numerice, text, binar - structuri), imagini, înregistrări audio și video, etc.
 - Structura fișierului depinde de tipul acestuia
- În cazul SO gen-UNIX se spune că
 - **Totul este fișier** – sau aproape (ce nu este fișier este proces)
 - Aceleași attribute și tipuri de operații asupra fișierelor se folosesc pentru gestiunea diferitelor interfețe ale SO

FIȘIERUL

- Unitatea logica de stocare este **fișierul**, iar unitate fizica de stocare este **blocul**
- Fișierul este un container de date
 - Unitatea în care datele stocate persistent sunt prezentate utilizatorului
 - Face legătura între viziunea utilizatorului despre date și modul de stocare efectiv pe dispozitive
 - Menține legătura dintre blocurile fizice distincte care conțin datele din fișier
- SO trebuie să furnizeze modalități pentru
 - Stocarea conținutului fișierului pe dispozitive
 - Regăsirea și accesarea ulterioară a conținutului
- Pentru a gestiona fișierele este nevoie de **metadata**
 - Informații care descriu fișierele stocate – pot fi stocate în zone diferite față de datele din fișier



ATRIBUTELE (METADATELE) FIȘIERELOR

- **Nume**
 - pentru referirea ușoară de către utilizatorii umani
- **Identificator**
 - Pentru a identifica în mod unic fișierul într-un SF
- **Tip**
 - Necesari în sistemele care suportă fișiere de tipuri diferite
- **Locație**
 - Pointer la un dispozitiv și la locația la care se află fișierul pe dispozitivul de stocare
- **Dimensiune**
 - Dimensiune (în octeți, cuvinte, blocuri) și posibil și dimensiunea maximă permisă
- **Drepturi de acces (protecție)**
 - Determină cine ce drepturi are asupra fișierului (r, w, x, etc.)
- **Timp, dată, identificare utilizator**
 - Timpul creării, a ultimei actualizări, etc. – necesar pt monitorizarea utilizării și pt măsuri de securitate
- Informațiile despre fișiere sunt stocate în structura de directoare
 - O **intrare în director** conține
 - Nume fișier + identificator
 - Identificatorul localizează celelalte atribute

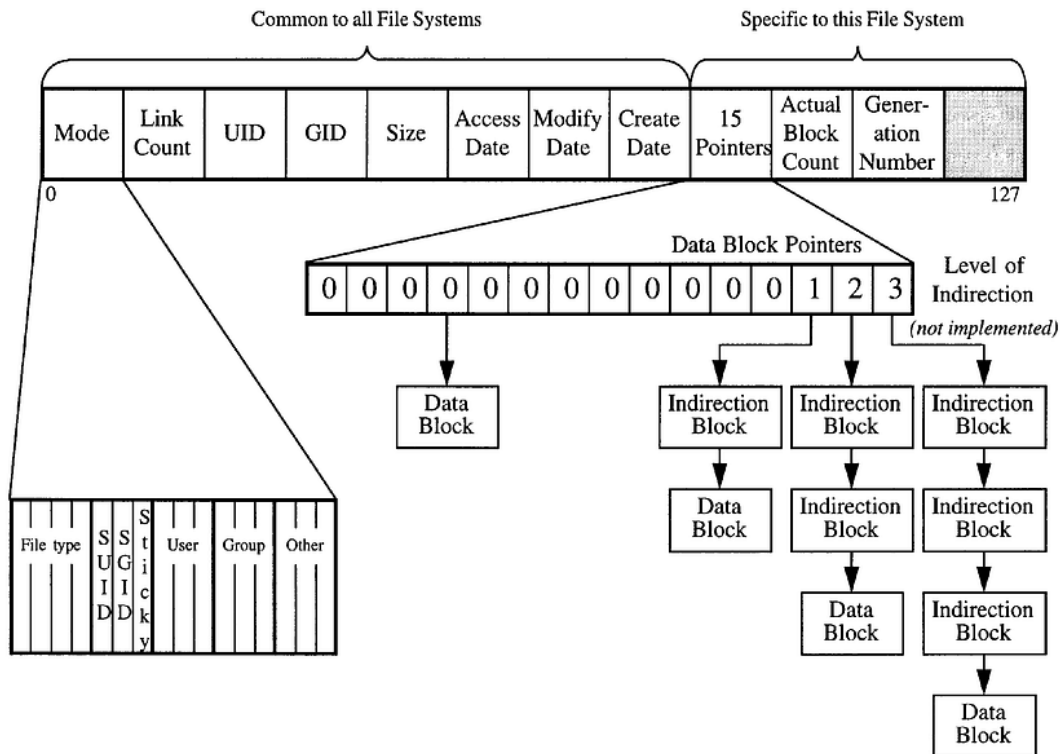
METADATA ȘI CONȚINUT

○ Inode – index node

- Index în tabela inode-urilor – identificatorul unic al fișierului / SF
- Structura care conține metadatele fișierului

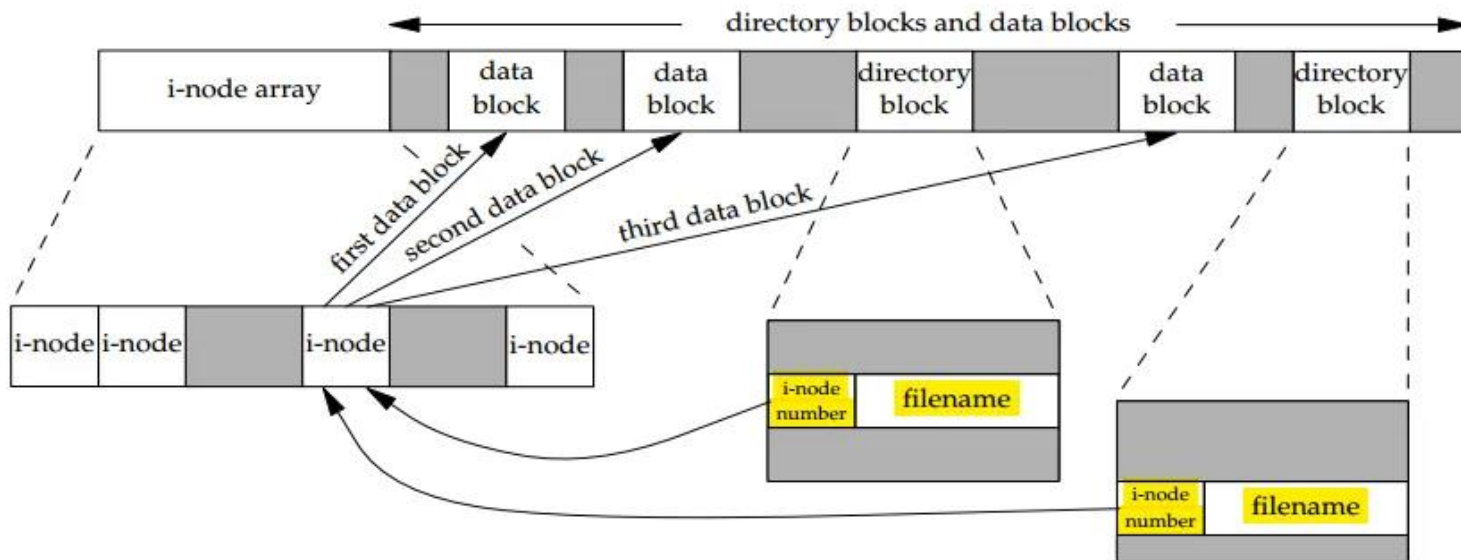
Disk Inode

<https://medium.com/@birnbera>



METADATA ȘI CONȚINUT

- Tabela de i-node-uri
 - Conține câte o intrare pentru fiecare fișier din sistem



Operații asupra fișierelor

- 6 operații fundamentale – set minim
 - Creare
 - Scriere
 - Citire
 - Repoziționare în cadrul fișierului
 - Ștergere
 - Trunchiere
- Alte operații obișnuite
 - Adăugare la final
 - Redenumire
 - Operații asupra atributelor

TABELA FIȘIERELOR DESCHISE

- Cele mai multe operații implică o **căutare** în director a intrării corespunzătoare numelui fișierului specificat
- Pentru evitarea căutărilor repetate – cele mai multe SO cer ca fișierul să fie **deschis** (open) înainte de folosirea activă

○ Tabela fișierelor deschise

- La efectuarea unei operații asupra fișierelor se specifică un index (descriptor de fișier) în această tabelă
 - nu se face căutare în structura de directoare

○ Definit per proces

- Câte o intrare pentru fiecare fișier deschis de proces
- Intrarea conține
 - Pointerul de poziție din cadrul fișierului, modul de accesare, etc.
 - Pointer către intrare în tabela fișierelor deschise per sistem

○ Definit per sistem

- **Intrarea conține** - informații independente de procesul care a deschis fișierul
 - LOCAȚIA UNDE ESTE STOCAT FIȘIERUL PE DISC, DIMENSIUNEA, DREPTURILE ETC.
 - NUMĂRUL DE PROCESE CARE AU DESCHIS FIȘIERUL
 - Fiecare închidere (close) descrescotează acest număr
 - când ajunge la 0 se șterge intrarea în tabelă

FIȘIERE DESCHISE

○ Informații necesare gestiunii fișierelor deschise

○ Cursorul fișierului (pointerul din fișier)

- Locația ultimei citiri sau scrieri
- Unic per descriptor

○ Contor de deschideri

- Numărul de deschideri efectuate pe fișier
- Se elimină intrarea în tabela fiș. deschise per sistem doar când acest contor este 0

○ Locația fișierului pe suport (disc)

- Informație de localizare pentru accesul la datele din fișier

○ Drepturi de acces

- Fiecare proces deschide fișierul într-un anumit mod de accesare
- Pe baza drepturilor specificate la creare fișierului (minus masca), SO poate permite sau refuza cereri de I/E

TABELA FIȘIERELOR DESCHISE

Figure 5-2 illustrates the relationship between file descriptors, open file descriptions, and i-nodes. In this diagram, two processes have a number of open file descriptors.

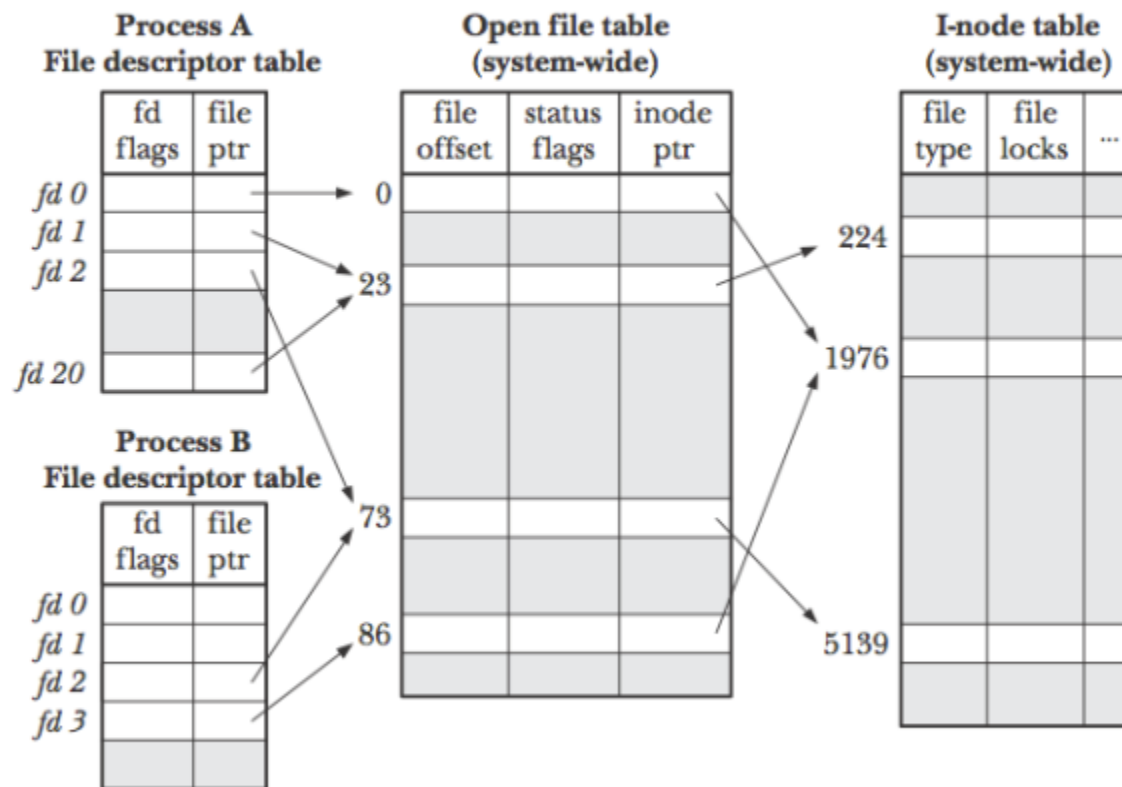


Figure 5-2: Relationship between file descriptors, open file descriptions, and i-nodes

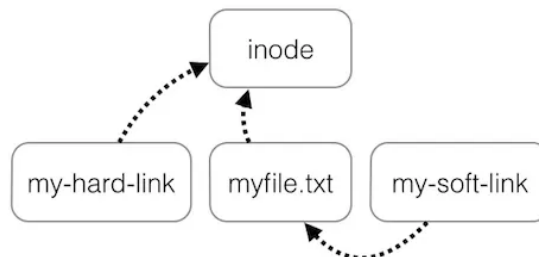
TIPURI DE FIȘIERE

- Fișiere **obișnuite** (ordinare, simple)

- **Directoare**

- Fișiere **legătură**

- Fizică
- Simbolică



- Fișiere **speciale** – modelează dispozitive de I/E

- Dispozitive de tip caracter (tastatură, mouse, etc.)
- Dispozitive de tip bloc (discuri)

- Fișiere **pipe** (pipe și fifo)

- Comunicare între procese

TIPURI DE FIȘIERE

○ Extensia

- Includerea tipului ca o parte a numelui
 - Determină **tipul de operații** care pot fi efectuate
- Tehincă obișnuită pentru implementarea tipurilor de fișiere
 - Convenție care furnizează **indicii asupra conținutului**
- De regulă nu este impusă de SO
- Poate fi recunoscută și folosită de SO
- Unele aplicații impun utilizarea anumitor extensii
 - Ex. gcc cere fișiere .c

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

STRUCTURA FIȘIERELOR

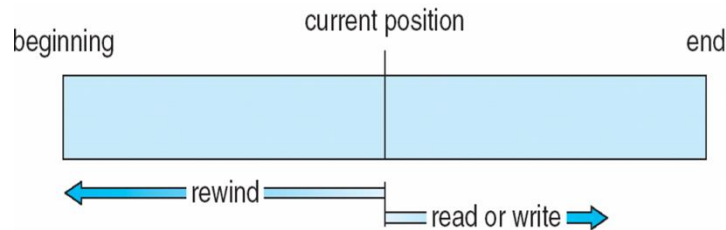
- Tipul fișierului poate indica structura internă
 - Tipuri diferite pot avea structuri diferite
- Nici o structură
 - Secvență de octeți
 - Flexibilitate maximă
 - Suport minim din partea SO
 - Fiecare aplicație trebuie să interpreteze fișierul conform cu structura așteptată
- Structură specifică
 - Secvență de structuri de dimensiune fixă
 - Ex. fișiere sistem – structura recunoscută de SO
 - Toate SO oferă suport pentru fișiere executabile
 - Necesită cod adițional pentru suport
 - Gestiunea se realizează în mod transparent utilizatorului
 - Cu set specific de operații
 - Dacă este impus
 - Toate fișierele trebuie să se încadreze într-una din categorii
 - Problema aplicațiilor care cer altă structură
 - Portabilitate redusă
 - structurile specifice nu pot fi menținute la trecerea pe alt SF

Dpdv al SO toate fișierele sunt doar o succesiune de octeți

METODE DE ACCES

○ Acces secvențial

- Informațiile din fișier sunt procesate în **ordine**
- Poate fi impus
 - De dispozitivul de stocare (ex. banda magnetică)
 - De natura conținutului (compilatorul parcurge fișierul în ordine, directorul este procesat în ordinea intrărilor)



○ Acces direct (sau aleator)

- Citirea și scrierea se efectuează **fără o ordine** specifică
- Fișierul este văzut ca o **secvență de blocuri** sau structuri
- Specific dispozitivelor care furnizează acces direct – discuri
- Includerea operației de mutare a cursorului la poziția specifică
 - Relativ la începutul fișierului

DIRECTOR

- **Fișier special** gestionat de SO în mod transparent utilizatorului
- Mod de **organizare** a fișierelor
 - Necesitatea stocării mai multor fișiere (mii și milioane)
- Conține
 - Fișiere și subdirectoare
 - **Intrări în director** – de regulă asocieri **nume - identificator**
- Poate fi organizat în diverse moduri – de regulă structură ierarhică
 - Facilitează regăsirea fișierelor – navigare în subdirectoare
- Operații
 - **Căutarea** unui fișier / subdirector
 - **Crearea** unui fișier / subdirector - adăugarea de intrări noi în director
 - **Ștergerea** unui fișier / subdirector - eliminarea unei intrări în director
 - **Listarea** conținutului – parcurgere secvențială
 - **Redenumirea** unui fișier / subdirector
 - **Traversarea** sistemul de fișiere

CĂI DE ACCES

- Furnizează modalitatea de a identifica fișierul în structura sistemului de fișiere
 - Noduri consecutive care se termină cu numele fișierului/directorului căutat
- Căi absolute
 - Nodul de pornire este întotdeauna rădăcina (root-ul)
 - Exemple
 - /home/so/curs3/program.c
 - C:\Users\Public\Documents
- Căi relative
 - Nodul de pornire este directorul curent
 - Fiecare aplicație are un director curent asociat
 - Noduri cu semnificație specială
 - . - directorul curent
 - .. - directorul părinte
 - Exemple
 - curs3/program.c
 - Users\Public\Documents

Operații asupra Directoarelor

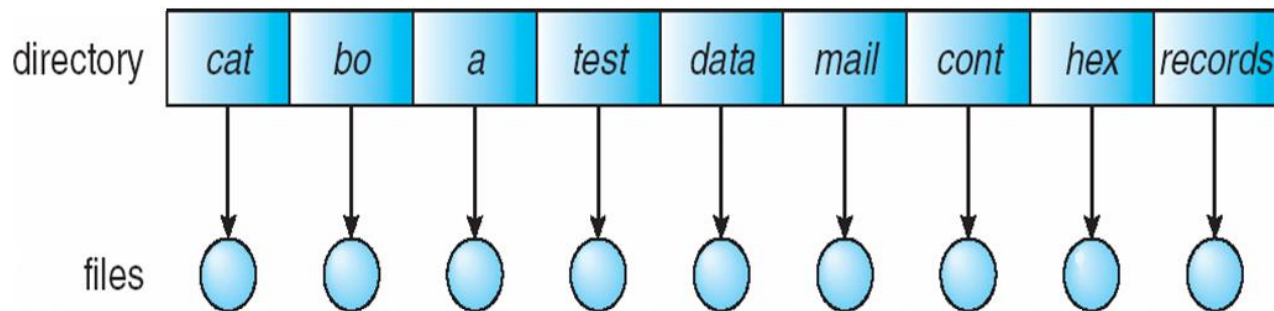
- create
- opendir
- readdir
 - Doar în acces secvențial
 - Poziționarea în locații arbitrare nu este permisă
- rewind
- write
 - Nu direct, ci prin adăugarea sau ștergerea de înregistrări în director
- close
- rename
- delete

STRUCTURA DE DIRECTOARE

- Colecție de directoare care conțin informații despre toate fișierele
- Structură pe un singur nivel
- Structură pe două niveluri
- Structură ierarhică

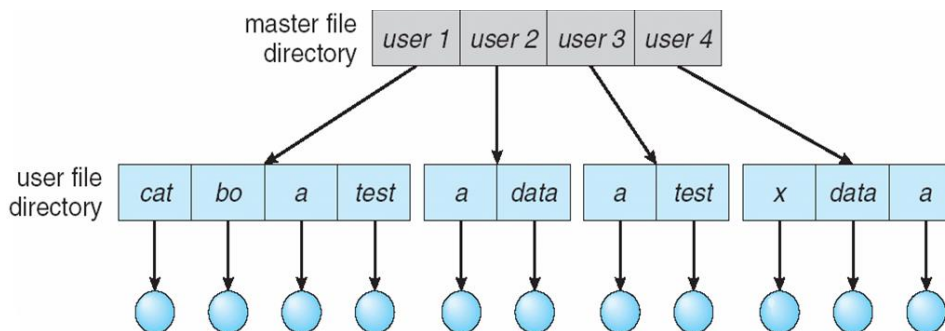
STRUCTURA DE DIRECTOARE PE UN SINGUR NIVEL

- Cea mai simplă
- Toate fișierele sunt cuprinse într-un **singur director**
- Limitări severe dacă sunt multe fișiere sau mai mulți utilizatori
- Problema **numelor unice**
- Problema **grupării fișierelor**



STRUCTURA DE DIRECTOARE PE DOUĂ NIVELURI

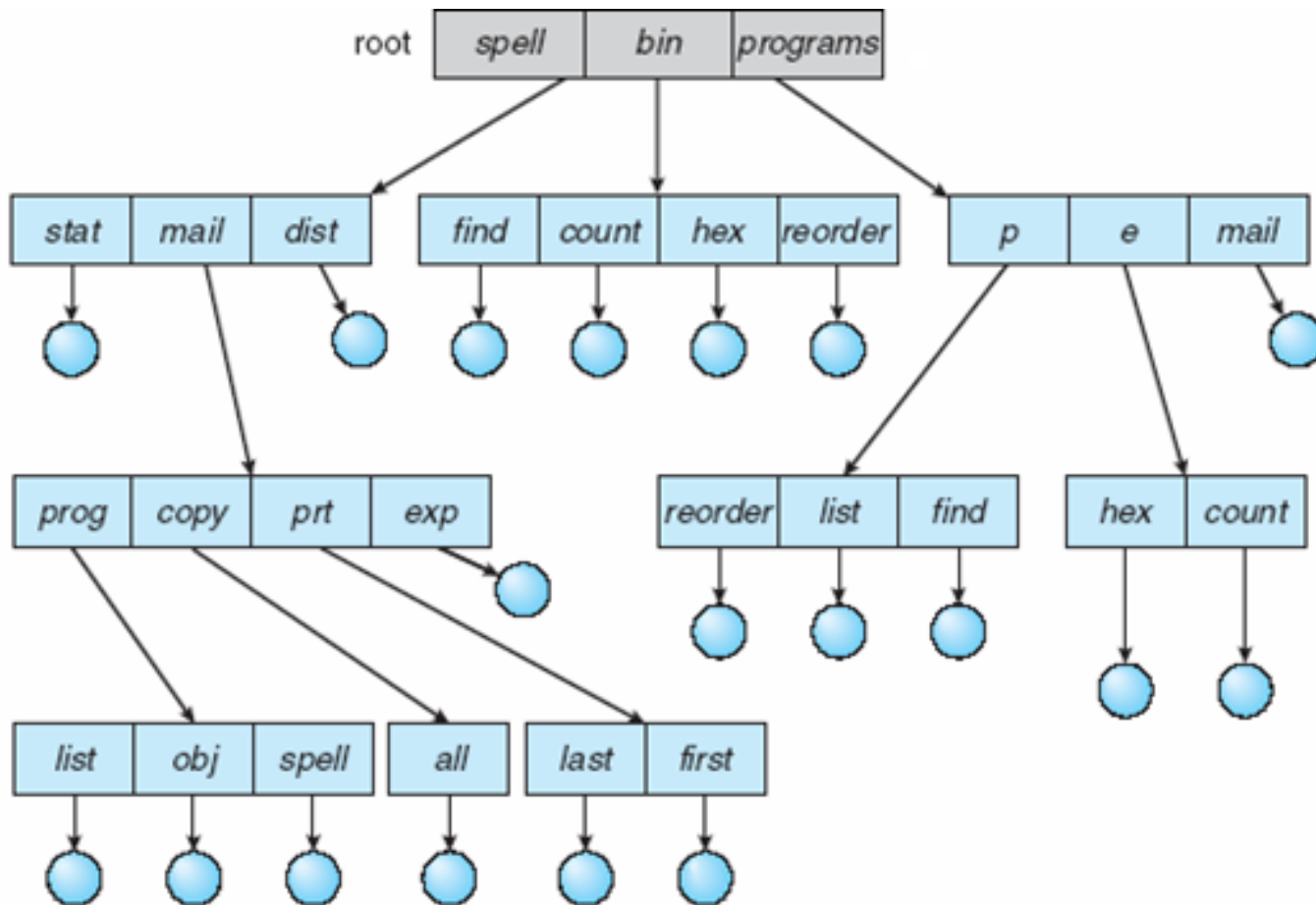
- **Directoare separate** pentru fiecare utilizator
 - UFD – User File Directory
 - MFD – Master File Directory
- **Izolarea** utilizatorilor
- Rezolvă problema **coliziunilor** de nume
 - Mai mulți utilizatori pot folosi același nume pentru fișierele lor – nume unice doar în cadrul unui UFD
- Căutare mai eficientă
 - Cale: nume utilizator + nume fișier
- Problema **grupării**



STRUCTURA DE DIRECTOARE IERARHICĂ

- Cea mai populară structură
- Sub formă de **arbore**
 - Număr arbitrar de niveluri
 - O rădăcină
- **Director curent**
 - Fiecare proces are un dir. curent
 - Poate fi schimbat
- **Căi de acces unice**
 - Cale absolută
 - Începând de la rădăcină
 - Cale relativă
 - Față de directorul curent
- Capacitate de **grupare**
 - Utilizatorul poate să-și structureze fișierele în subdirectoare
- Căutare eficientă

STRUCTURA DE DIRECTOARE IERARHICĂ



PROTECȚIE

○ Siguranță

- Față de defect fizic – **fiabilitate**
- Față de acces neautorizat – **protecție**

○ Acces controlat

- Proprietarul fișierului poate decide cine ce drepturi are asupra fișierului
 - **Ce se poate face**
 - **De către cine**
- Mecanisme variate

○ Tipuri de acces

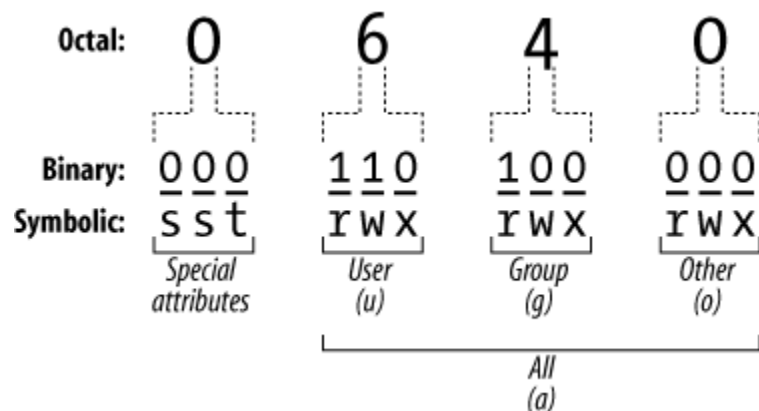
- Citire
- Scriere
- Execuție
- Adăugare
- Ștergere
- Listare

CONTROLUL ACCESULUI

- Permite acces pe baza **identității utilizatorului**
- **Listă de control al accesului** (ACL – Access Control List)
 - Asociat fiecărui fișier și director
 - Lista utilizatorilor și tipul accesului permis pt fiecare
 - Avantaj
 - Permite metodologii de acces complexe
 - Dezavantaj
 - Dimensiunea listei, cunoștințe despre utilizatori -> practicalitate redusă
 - **Clasificarea utilizatorilor** – trei categori
 - Proprietar – user (u)
 - Grup – group (g)
 - Alții – others (o)
- ACL pentru categorii de utilizatori

DREPTURI DE ACCES

- Operații
 - Citire – read (r)
 - Scriere – write (w)
 - Execuție – execute (x)
- Definite pe trei categorii de utilizatori
 - Proprietarul (u - user)
 - Grupul din care face parte user-ul (g - group)
 - Alții (o - others)
- Structura permisiunilor



EXEMPLU DE LISTARE A UNUI DIRECTOR ÎN UNIX

-rw-rw-r--	1	pbg	staff	31200	Sep 3 08:30	intro.ps
drwx-----	5	pbg	staff	512	Jul 8 09:33	private/
drwxrwxr-x	2	pbg	staff	512	Jul 8 09:35	doc/
drwxrwx---	2	pbg	student	512	Aug 3 14:13	student-proj/
-rw-r--r--	1	pbg	staff	9423	Feb 24 2003	program.c
-rwxr-xr-x	1	pbg	staff	20471	Feb 24 2003	program
drwx--x--x	4	pbg	faculty	512	Jul 31 10:31	lib/
drwx-----	3	pbg	staff	1024	Aug 29 06:52	mail/
drwxrwxrwx	3	pbg	staff	512	Jul 8 09:35	test/

DREPTURI DE ACCES

- Se referă la acțiunea care poate fi efectuată asupra **conținutului**

- Asupra **fișierelor** obișnuite
 - Citire
 - Citirea conținutului – afișarea
 - Scriere
 - Modificarea conținutului – adăugare, schimbare, trunchiere
 - Execuție
 - Execuția instrucțiunilor din fișier

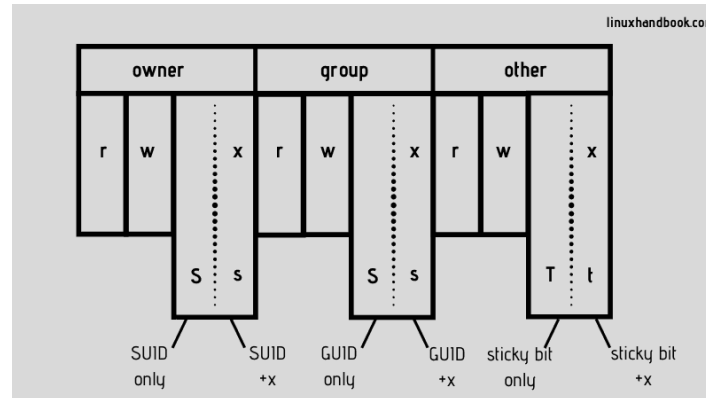
- Asupra **directoarelor**
 - Citire
 - Listarea conținutului – intrărilor în director (fișiere, subdirectoare)
 - Scriere
 - Modificarea conținutului – creare de noi fișiere și subdirectoare, redenumire de fișiere și subdirectoare, ștergere de fișiere și subdirectoare
 - Execuție
 - Traversarea directorului pentru a accesa fișiere și subdirectoare

DREPTURI DE ACCESS - SPECIALE

o Setuid - SUID

- o Apare **s** (mic) în locul **x**-ului la proprietar (ex. -rw**s**r-xr-x)
- o Fișierul executabil rulează cu drepturile specifice proprietarului fișierului și nu cu privilegiile utilizatorului care lansează în execuție
 - Exemplu binecunoscut: schimbarea parolei proprii folosind executabilul **passwd** al cărui proprietar este root-ul

```
chmod u+s file_name  
chmod 4766 test2.txt
```



o Sticky bit

- o Se aplică doar asupra directoarelor
- o Fișierele din director pot fi modificate doar de către proprietarul lor
 - Exemplu binecunoscut: directorul /tmp (ex. drwxrwxrwt)
 - o Poate fi scris de orice utilizator, dar ca să prevină modificarea sau ștergerea fișierului de alt utilizator, se setează bitul sticky

```
chmod g+s folder  
chmod 2775 folder2
```

```
chmod +t my_dir
```

```
chmod 1775 tmp2/
```

o Setguid - SGID

- o Se aplică asupra fișierelor și directoarelor
- o Fișierul executabil rulează cu drepturile specifice grupului din care face parte proprietarului fișierului și nu cu privilegiile grupului utilizatorului care lansează în execuție
- o Dacă este setat pe director, fișierele create înăuntru vor avea grupul proprietarului directorului, indiferent de proprietarul fișierelor (ex. drwxrw**s**r-x)
 - Folosit la partajarea fișierelor

FIȘIERE LEGĂTURĂ

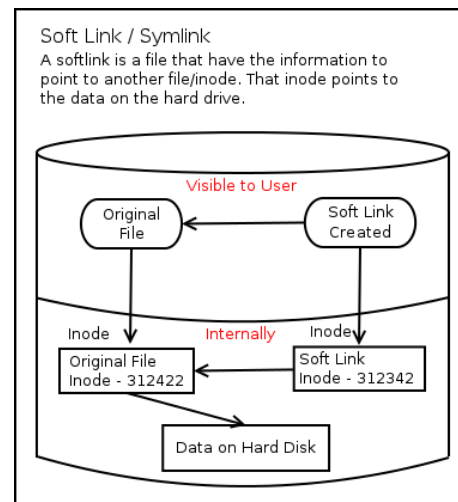
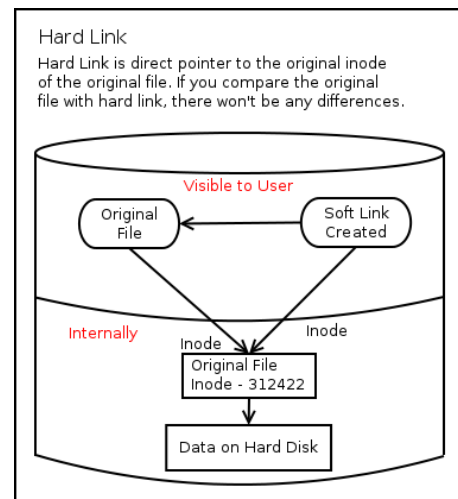
- Permite accesul la același fișier din locații (directoare) diferite

- **Legătură fizică (hard link)**

- Se creează doar o nouă intrare în director
 - Referă același inode
 - Se incrementează numărul de legături din metadate
 - La ștergerea fișierului original se decrementează contorul conținutului nu se șterge, decât la contor = 0
 - Doar pt fișiere
 - Doar pe același SF

- **Legătură simbolică (soft / symbolic link)**

- Se creează un nou inode + o nouă intrare în director
 - Fișierul legătură conține calea către fișierul legat
 - La ștergerea/relocarea fișierului original legătura devine invalidă
 - Permite legătură între SF diferite
 - Permite legătură între directoare



IMPLEMENTAREA SISTEMULUI DE FIȘIERE

STRUCTURA SISTEMULUI DE FIȘIERE

○ Fișier

- Unitate logică de stocare
- Colecție de informații înrudite

○ SF rezidă pe spațiul de stocare secundar – disc

- Furnizează **interfață** utilizator pentru stocare
 - Maparea spațiului logic de blocuri la spațiul fizic
- Furnizează **acces eficient** la datele stocate pe disc

○ Decizii de proiectare

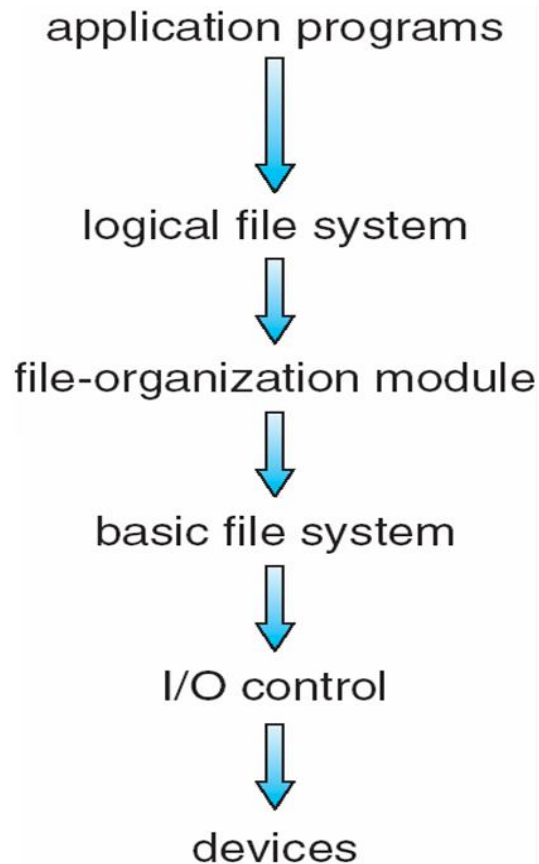
- Cum arată **interfața** sistemului de fișiere
 - Atribute, operații, structura de directoare
- Decizii de **implementare**
 - Structuri de date și algoritmi pentru maparea spațiului logic la cel fizic

○ Bloc de control al fișierului (FCB – File Control Block)

- Structură de date care conține informații despre fișier
 - Ex. Structura i-node

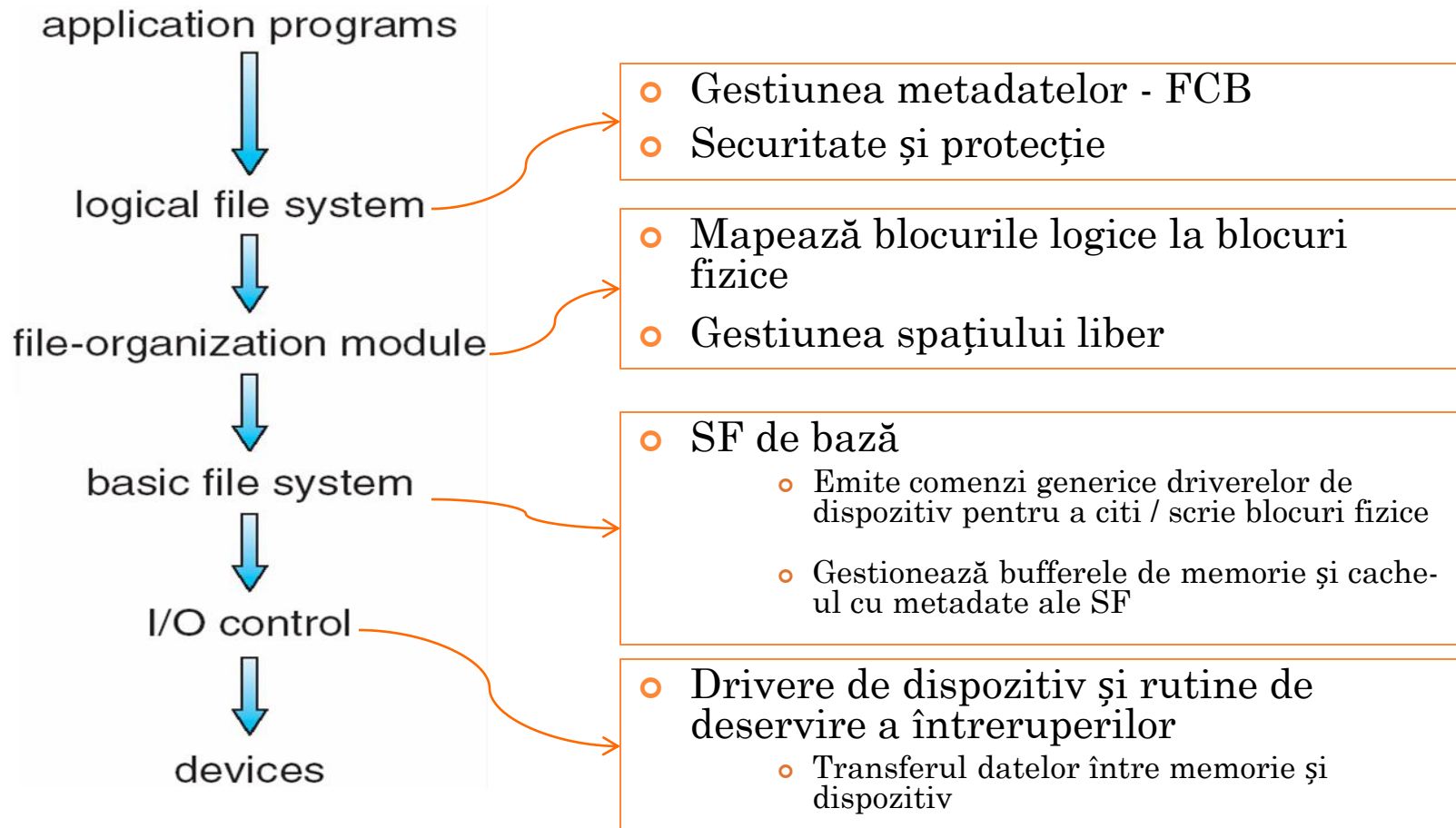
NIVELURILE SF

- Sistemul de fișiere este organizat pe niveluri
 - Fiecare nivel utilizează caracteristicile nivelului inferior pentru a crea funcționalități pentru nivelul superior



NIVELURILE SF

- Sistemul de fișiere este organizat pe niveluri
 - Fiecare nivel utilizează caracteristicile nivelului inferior pentru a crea funcționalități pentru nivelul superior



IMPLEMENTAREA SF

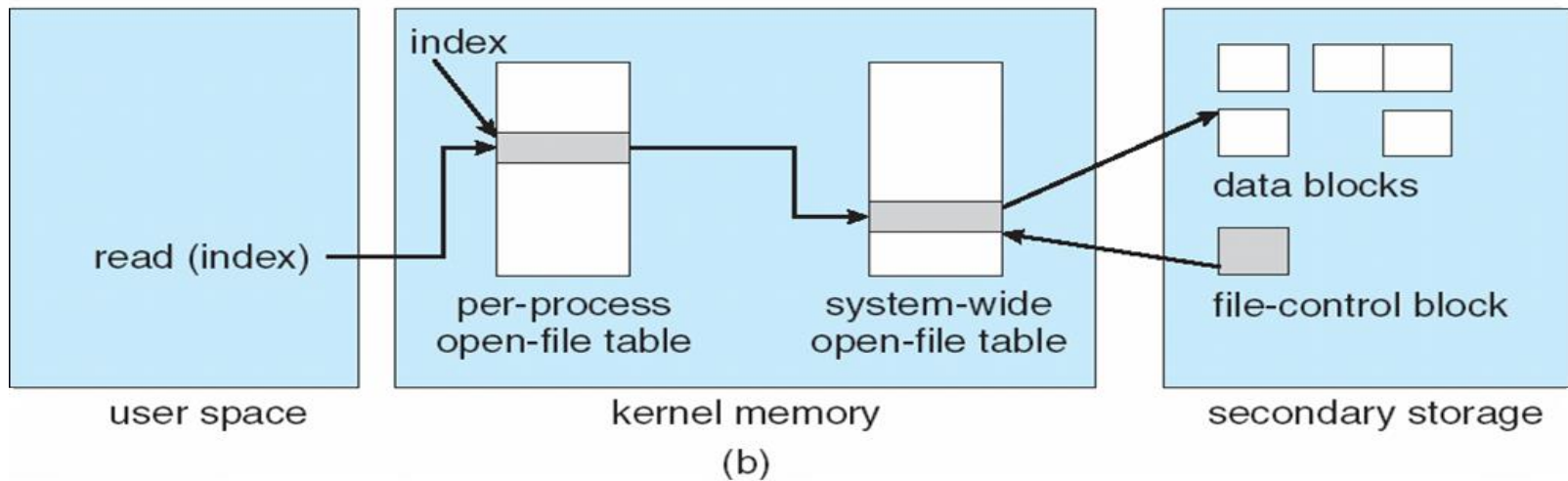
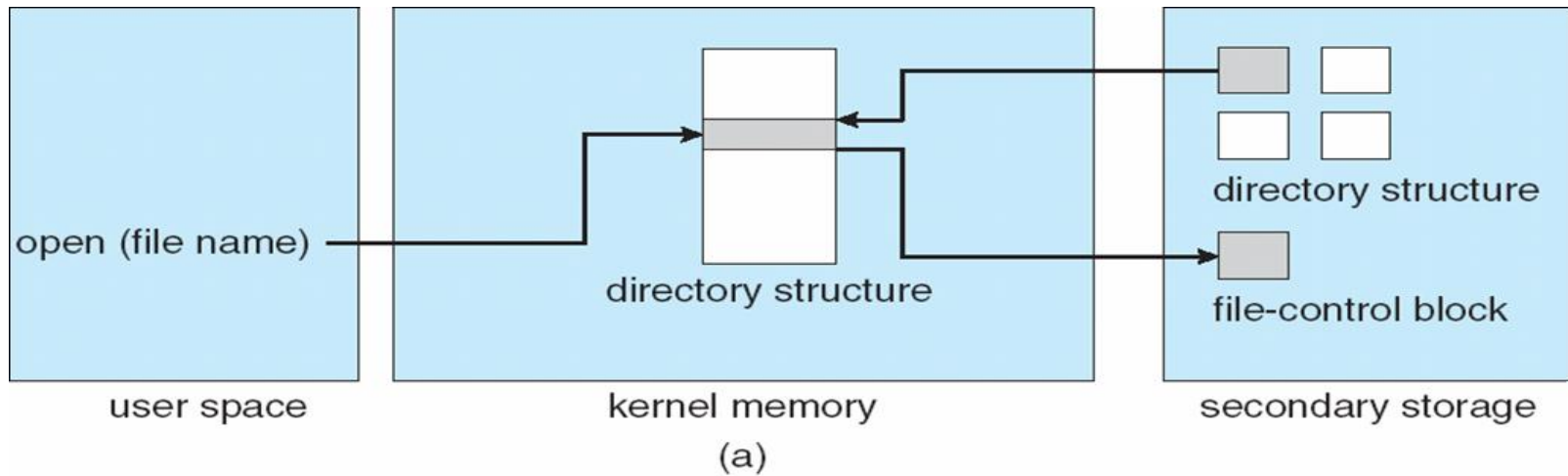
- **Structurile de date** folosite pentru implementarea operațiilor sistemului de fișiere
- Implementări variate dar principiile de bază sunt comune
 - **Structuri stocate pe disc**
 - Bloc de control pentru pornirea SO (per volum)
 - Informații necesare pentru încărcarea SO de pe volum
 - Numit **boot block** sau **partition boot sector**
 - Bloc de control al volumului (per volum)
 - Informații despre partiție
 - Număr de blocuri , dimensiunea unui bloc, Contorul blocurilor libere și al FCB-urilor libere, Pointer la blocurile libere și la FCB-urile libere
 - Numit **superblock** sau **master file table**
 - **Structura de directoare** (per SF)
 - Pentru organizarea fișierelor
 - **FCB** pentru fiecare fișier
 - Conține informații despre fișier – attribute, metadata (i-node)

IMPLEMENTAREA SF

◦ Structuri menținute în memorie

- Pentru **gestiunea SF** și **optimizarea** performanței prin caching
 - Informații sunt
 - ÎNCĂRCATE LA MONTARE
 - ACTUALIZATE ÎN TIMPUL OPERAȚIILOR ASUPRA SF
 - ELIMINATE LA DEMONTARE
- **Tabela de montare**
 - Informații despre fiecare volum montat
- **Structura de directoare - cache**
 - Informații despre directoare accesate recent
- **Tabela fișierelor deschise per sistem**
 - Include copii ale FCB-urilor pentru fiecare fișier deschis
- **Tabela fișierelor deschise per proces**
 - Conține un pointer la intrarea corespunzătoare din tabela globală și alte info
- **Buffere**
 - Conțin date în transfer de la disc sau spre disc

STRUCTURI MENȚINUTE ÎN MEMORIE



IMPLEMENTAREA DIRECTOARELOR

○ Alocarea & gestiunea directoarelor

- Decizie care afectează eficiența, performanța și fiabilitatea SF

○ Listă liniară

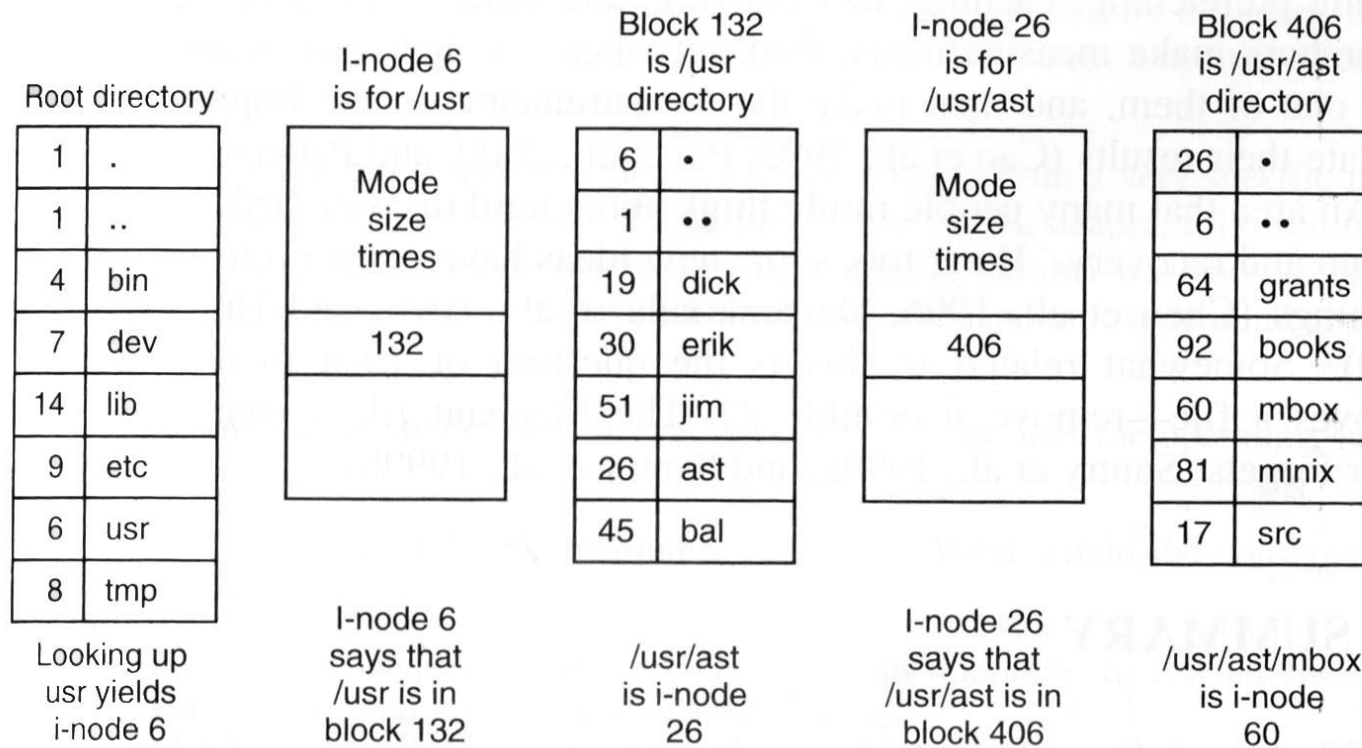
- Listă liniară de fișiere cu pointeri către blocurile de date
- Implementare ușoară
- Timp mare de execuție
 - Regăsirea unui fișier necesită căutare liniară
- Soluții
 - Ordonare alfabetică
 - Poate complica crearea și ștergerea
 - Implementarea ca arbori (Arbori B)

○ Tabele de dispersie

- Listă liniară + structură de dispersie
 - returnează un pointer către numele fișierului din lista liniară
- Timpul de căutare mult diminuat
- Problema coliziunilor

IMPLEMENTAREA DIRECTOARELOR ÎN UNIX

- Listă liniară de fișiere / subdirectoare și număr i-node
- Căutarea `/usr/ast/mbox`



METODE DE ALOCARE

- Provocare

- Alocarea spațiului pentru fișiere astfel încât
 - Discul să fie **utilizat eficient**
 - **Accesul** la date să fie **rapid**

- Trei metode

- **Alocare contiguă**
- **Alocare prin înlănțuire**
- **Alocare prin indexare**

ALOCARE CONTIGUĂ

- Fiecare fișier ocupă un set de **blocuri succesive**

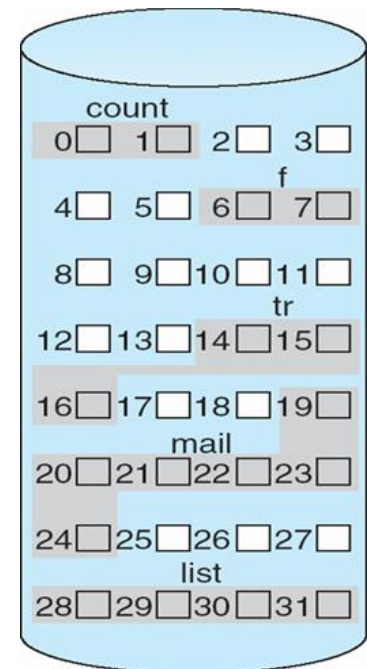
- Cea mai bună performanță în majoritatea cazurilor
 - Timpul de căutare se reduce la minim

- Metodă **simplă**

- Locația fișierului este determinată de
 - adresa blocului de început și
 - dimensiunea în număr de blocuri
- Suport pentru acces secvențial și direct

directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2



ALOCARE CONTIGUĂ

○ Exemplu

Contiguous File Allocation example



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Create **File1** with 6 blocks, starts at 1
Create **File2** with 16 blocks, starts at 7
Create **File3** with 38 blocks, starts at 23

ALOCARE CONTIGUĂ

○ Exemplu

Contiguous File Allocation example

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Create File1 with 6 blocks, starts at 1
Create File2  16 blocks, starts at 7
Create File3 with 38 blocks, starts at 23
Delete File2 Just delete the pointer

ALOCARE CONTIGUĂ

○ Exemplu

Contiguous File Allocation example



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Create **File1** with 6 blocks, starts at 1

Create **File3** with 38 blocks, starts at 23

Create **File4** with 7 blocks, starts at 7

ALOCARE CONTIGUĂ

○ Exemplu

Contiguous File Allocation example

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Create **File1** with 6 blocks, starts at 1

Create **File3** with 38 blocks, starts at 23

Create **File4** with 7 blocks, starts at 7

Edit **File1** Add about 15 blocks to it! i.e. delete old File1 & create new File 1

ALOCARE CONTIGUĂ

○ Exemplu

Contiguous File Allocation example



1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Create **File3** with 38 blocks, starts at 23

Create **File4** with 7 blocks, starts at 7

Edit **File1** Add about 30 blocks to it! i.e. delete old File1 & create new File 1, starts at 61

Delete **File1**

Create new **File1**

ALOCARE CONTIGUĂ

○ Exemplu

Contiguous File Allocation example

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Create File3 with 38 blocks, starts at 23

Create File4 with 7 blocks, starts at 7

Edit File1 with 30 blocks, starts at 61

Create File5 with 10 blocks, CANNOT be inserted!

X CANNOT CREATE File5 since there are no spaces with 10 contiguous blocks free
? But we have 19 blocks free on this drive/tape?

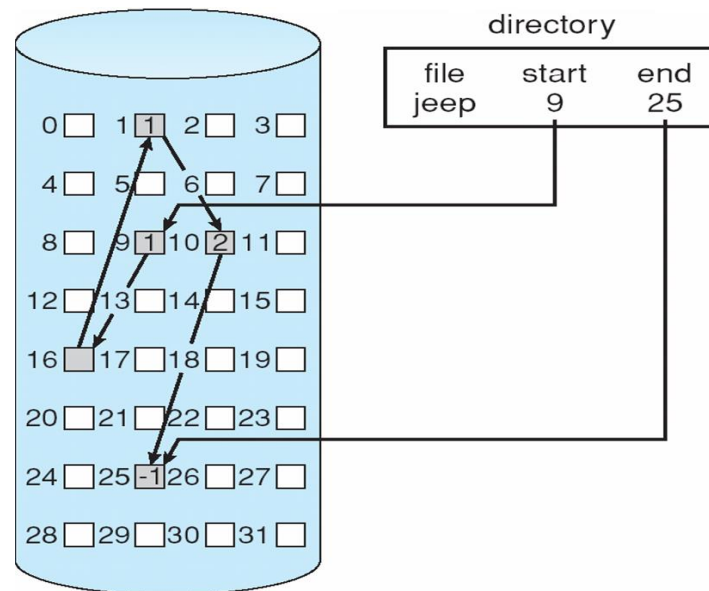
ALOCARE CONTIGUĂ

○ Probleme

- Găsirea spațiului liber de dimensiuni corespunzătoare
 - Metode: best-fit, next-fit, etc.
- Fragmentare externă
 - Necesită compactare – defragmentarea este costisitoare!
- Dimensiunea finală a fișierului nu este cunoscută la creare
- Dimensiunea fișierului se poate modifica în timp
 - Creșterea dimensiunii este condiționată de existența blocurilor libere în zona adiacentă
 - Soluție
 - bazată pe ștergere și recreare sau
 - bazată pe extensii

ALOCARE PRIN ÎNLĂNȚUIRE

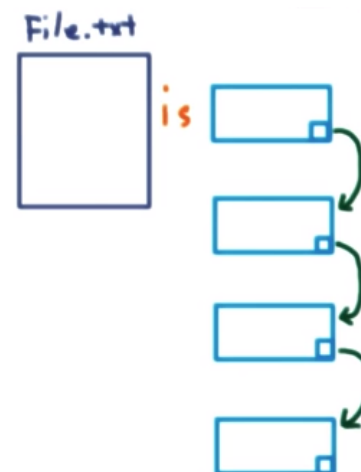
- Fiecare fișier este o **listă înlănțuită de blocuri fizice**
 - Blocurile pot fi dispersate pe disc
- Intrarea în director
 - Pointer către primul și ultimul bloc din listă
 - Blocurile conțin pointeri către următorul bloc
- **Elimină fragmentarea externă**
 - Deci și necesitatea de compactare



ALOCARE PRIN ÎNLĂNȚUIRE

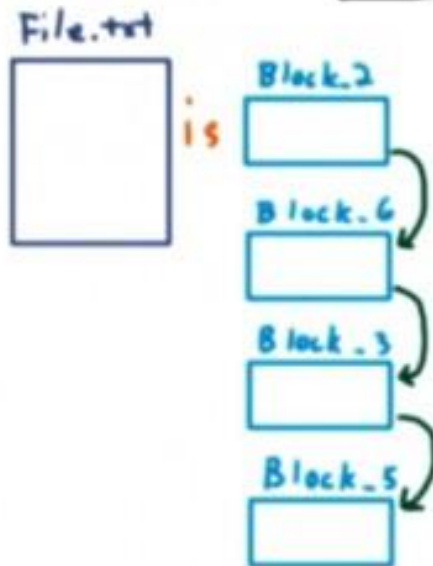
- **Dezavantaje**

- Eficient doar în cazul fișierelor cu **acces secvențial**
 - Repoziționarea directă necesită multe operații de i/e
- **Pointerul** ocupă spațiu de stocare
 - Soluție parțială: alocarea unor clustere (colecție de blocuri)
- Fiabilitatea este slabă
 - **Pierderea unui pointer** -> următoarele blocuri devin inaccesibile
 - Soluție parțială: liste dublu înlanțuite



FAT – FILE-ALLOCATION TABLE

File Allocation Table



	<u>FAT</u>	
	Busy	Next
0	0	
1	1	-1
2	1	6
3	1	5
4	1	-1
5	1	-1
6	1	3
7	0	

Directory Table Former

/

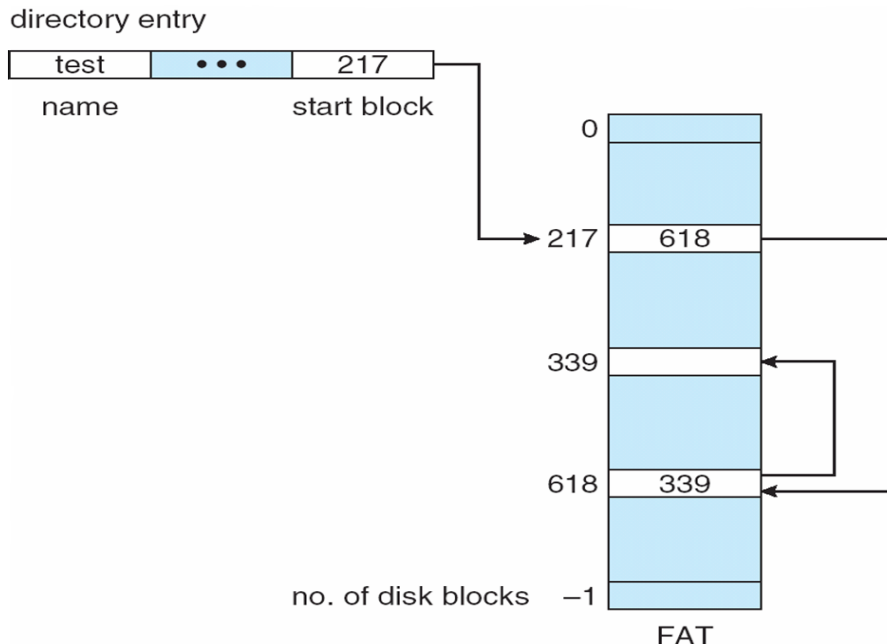
filename	starting block	meta data
foo	1	

/foo

filename	starting block	meta data
File.txt	2	

FAT – FILE-ALLOCATION TABLE

- O variantă de alocare înlănțuită – mult mai eficientă
- O parte de disc de la începutul fiecărui volum este rezervat pentru FAT
 - Câte o intrare pentru fiecare bloc
- Intrarea în director conține numărul primului bloc ocupat de fișier
 - Intrarea în tabelă de la indexul blocului conține numărul următorului bloc șamd.
 - Ultimul bloc are valoarea end-of-file
 - Blocuri libere – valoare 0

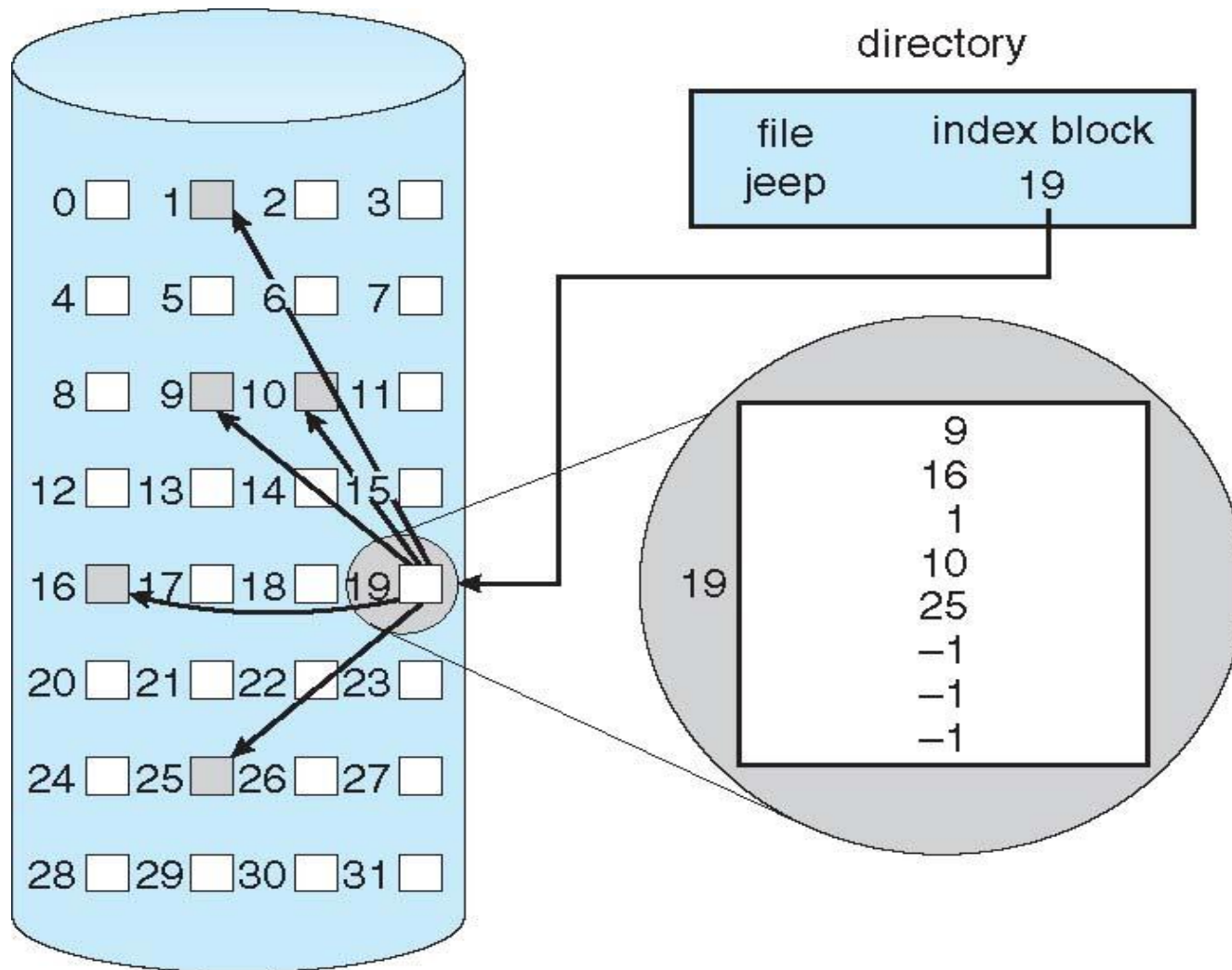


- Poate rezulta în număr mare de mutări ale capului de citire a discului
 - Soluție cache al FAT-ului
- Timpul de acces direct este mult îmbunătățit
 - Adresa fiecărui bloc poate fi găsită consultând FAT-ul

ALOCARE PRIN INDEXARE

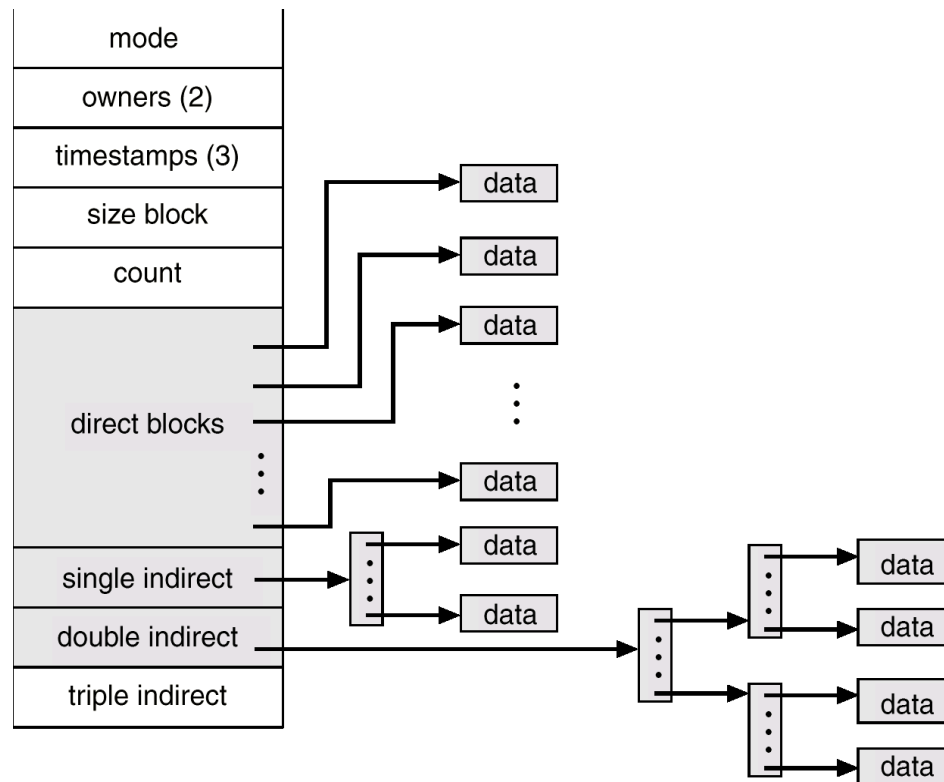
- Alocarea prin înlănțuire
 - Rezolvă unele probleme ale alocării contigue
 - fragmentarea externă și cele legate de dimensiunea fișierelor
 - În absența FAT-ului nu oferă suport pentru accesare directă eficientă
- Alocarea prin indexare rezolvă această problemă
 - Aduce toți pointerii la un loc – **bloc de indexuri**
- Fiecare fișier are propria tabelă de indexuri
- Avantaj
 - Suport pentru **accesare directă fără fragmentare externă**
- Probleme
 - **Spațiu irosit** în stocarea pointerilor (se alocă un bloc fizic)
- Metode de implementare a tabelii de indexuri pt fișiere mari
 - Schemă înlănțuită
 - Schemă multinivel
 - Schemă combinată

ALOCARE PRIN INDEXARE



SCHEMĂ COMBINATĂ

- Folosită de UFS (Unix File System)
 - Primele 12 indexuri pointează direct la blocuri de date
 - Următoarele 3 indexuri pointează indirect la blocuri
 - Primul cu un singur nivel de indirectare
 - Al doilea cu două niveluri de indirectare, etc.



PERFORMANȚA

- Depinde de modul de accesare
- **Alocarea contiguă**
 - potrivită pentru acces secvențial și aleator (direct)
- **Alocarea prin înlănțuire**
 - Potrivită pentru accesare secvențială
- **Alocarea prin indexare**
 - Complexă
 - Un singur acces la un bloc poate necesita două citiri din tabela indexurilor + citirea blocului
 - Clusterizarea poate îmbunătăți performanța
- => Alocarea folosind extensii**
 - În loc de blocuri individuale, referă clustere de blocuri consecutive (ext4, NTFS)

FRAGMENTARE

○ Fragmentare externă

- Zone nealocate de dimensiuni mici distribuite pe disc
- Problemă la alocarea unei zone contigue
 - Nici una din zonele contigue libere nu e suficient de mare
 - Deși per total este suficient spațiu liber
- Soluție costisitoare: defragmentarea
 - Gruparea zonelor libere într-o zonă contiguă

○ Fragmentarea internă

- Apare în cazul alocării în blocuri de dimensiune fixă
 - rămâne zonă neutilizată în cadrul blocului alocat
- Soluție – alocare în blocuri de dimensiuni mai mici

○ Fragmentarea datelor

- Conținutul fișierului este distribuit în blocuri neconsecutive
- Reduce performanța în lucrul cu fișierul
- Soluție: defragmentarea
 - Gruparea conținutului în zone contigue

DIMENSIUNEA BLOCULUI DE ALOCARE

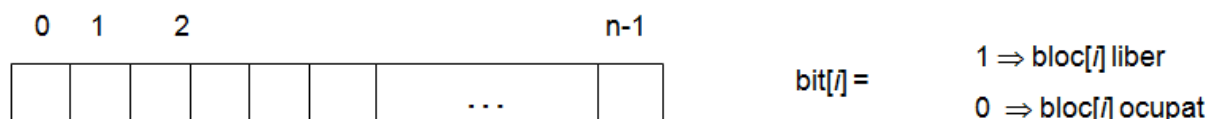
- Performanța vs. utilizarea eficientă a spațiului
 - Ce dorim să maximizăm?
 - Alegere pe baza unui compromis potrivit contextului
- Blocuri mai mici
 - Fragmentare internă mai redusă
 - Multiple accesări ale discului
 - Fragmentare mai mare a datelor
- Blocuri mai mari
 - Fragmentare internă mai mare
 - Acces facil la date – citire cu mai puține accesări al HDD

PERFORMANȚA COMPARATIVĂ – CPU vs I/O

- Intel Core i7 Extreme Edition 990x (2011) la 3.46GHz = 159000 MIPS
- Un disc obișnuit la 250 I/O per sec
 - $159000 \text{ MIPS} / 250 = 630$ milioane de instrucțiuni pe durata unei op I/O
- Dispozitive SSD (Solid State Drive) furnizează 60000 IOPS
 - $159000 \text{ MIPS} / 60,000 = 2.65$ milioane de instrucțiuni pe durata unei op I/O

GESTIUNEA SPAȚIULUI LIBER

- Spațiu limitat -> reutilizare
- SF menține o **listă de blocuri libere** pe disc
 - **Hartă de biți sau Vector de biți (bit map)**



- Calcularea poziției primului bloc liber
 - nr. biți per cuvânt *
nr. cuvinte cu val. 0 +
deplasamentul primului bit cu val. 1
- **Avantaj**
 - Metodă relativ simplă - ușurință în găsirea unui spațiu contiguu
- Dezavantaj
 - Menținerea în memorie a unei copii a hărții de biți- necesită spațiu adițional

Apelează la instrucțiuni specifice care returnează deplasamentul primului bit cu val 1 dintr-un cuvânt

Exemplu:

dimensiune bloc = 4KB = 2^{12} B

dimensiune disc = 2^{40} B (1 terabyte)

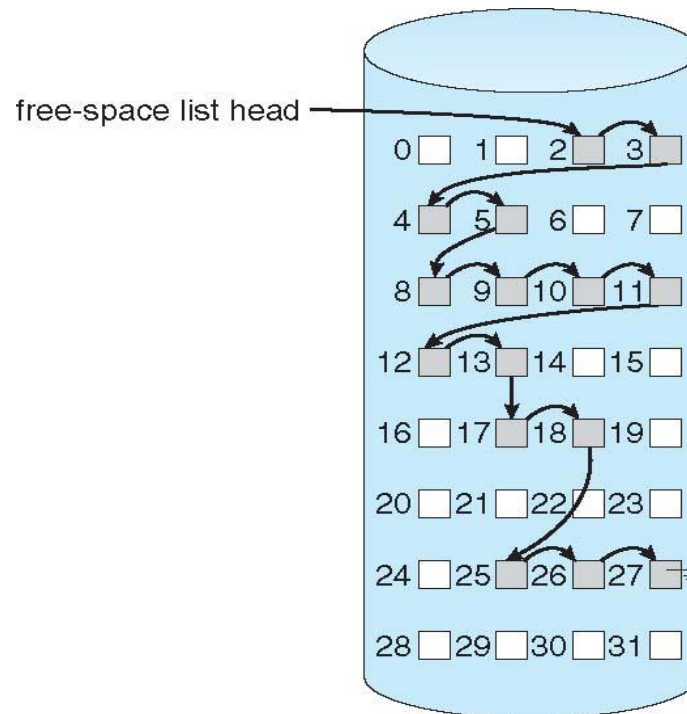
$n = 2^{40}/2^{12} = 2^{28}$ bits (sau 256 MB)

dacă se grupează în clustere de câte 4 blocui -> 64MB de memorie

GESTIUNEA SPAȚIULUI LIBER

- Listă înlănțuită

- Menține în memorie doar pointerul către primul bloc liber
- Ineficient pentru traversare
 - Dar de regulă nu este necesară traversarea
- Prin metoda FAT se face atât alocare cât și evidența spațiului liber
- Regăsirea unui spațiu liber contiguu este mai costisitoare



GESTIUNEA SPAȚIULUI LIBER

○ Grupare

- Variantă a metodei cu liste înlănțuite
- Stochează adresele primelor $n-1$ blocuri libere în cadrul primului bloc liber
 - + un pointer către următorul bloc care conține adrese de blocuri libere

○ Contorizare

- Util mai ales în cazul alocării contigue
 - Dar și la alocare în clustere, sau cu extensii
- Menține adresa primului bloc liber și a numărului de blocuri libere succesive
 - Lista blocurilor libere conține în intrări atât adresa blocului cât și contorul

VA URMA ... 😊